

agents vs agentic ai -

While **AI agents** are individual, modular software entities designed to perform narrow, specific tasks using tools, **agentic AI** is the broader system architecture and operational paradigm where multiple agents, data sources, and reasoning loops are coordinated to manage complex, end-to-end workflows

Claude Code is the **agent harness**; when combined with a frontier Claude model, it creates an **agentic AI** software engineering system.

Agent Harness (The Execution & Control System): It is the runtime environment. It contains set of tools and the environment in which it executes. This includes the execution loop (perceive -> decide -> act -> observe), tool integrations (file editing, terminal commands, APIs), memory persistence, and security/permission checks.

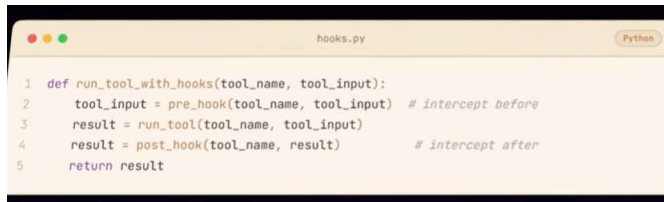
Agentic coding is the application of agentic AI principles specifically to software development—moving from passive code generation to autonomous, goal-driven software engineering

Agent harness + LLM = Agentic AI

Loop Phase	Primary System	Function
Observe	Harness	Captures raw environmental responses, errors, and status codes.
Perceive	Harness -> LLM	Harness turns observe raw data into context- suitable for the context window; Reads, parses, and comprehends the formatted context provided by the harness to update its internal context model.
Decide	LLM	Synthesizes context to select the next tool or text response.
Act	Harness	Intercepts tool calls and executes them in the external runtime.

1. **You** send a message ("Create a new file with a hello world function")
2. **The LLM** decides it needs a tool and responds with a structured tool call (or multiple tool calls)
3. **Your program** executes that tool call locally (actually creates the file)
4. **The result gets sent back to the LLM**
5. **The LLM** uses that context to continue or respond

```
# n0 master loop
while(tool_call):
    execute_tool()
    feed_results_to_model()
    repeat()
```



```
hooks.py Python
1 def run_tool_with_hooks(tool_name, tool_input):
2     tool_input = pre_hook(tool_name, tool_input) # intercept before
3     result = run_tool(tool_name, tool_input)
4     result = post_hook(tool_name, result) # intercept after
5     return result
```

The harness spans subagent using another tool call. So spanning subagent is also just a tool call

The Anthropic Claude API separates inputs into **system** and **user** prompts to distinguish high-level developer instructions from runtime end-user input.

System Prompt: Defines the baseline rules, persona, formatting constraints, available tools, and guardrails set by the developer/operator. -- Also Prompt cacheing becomes better.

User prompt : What user entered.